BINUS UNIVERSITY ONLINE

People
Innovation
Excellence

Tugas Kelompok ke-3

Week 8

Algorithmic Analysis, Mathematical Induction, and Recursive Functions

Learning Outcome:

- LO2: Analyze algorithm complexity: Students will be able to assess the time and space complexity of algorithms

---

Instruksi Pengerjaan

1. Buat program menggunakan Java dengan nama file PlaylistRekursif.java.
2. Gunakan array objek Lagu (dari tugas sebelumnya) untuk menyimpan data playlist.
3. Implementasikan fungsi-fungsi berbasis rekursif untuk melakukan operasi analisis pada playlist.
4. Tulis komentar di setiap fungsi utama untuk menjelaskan:
   - Tujuan fungsi
   - Base case dan recursive case
   - Kompleksitas waktunya
5. Tambahkan fitur pengukuran waktu eksekusi (time measurement) yang ada dalam Java.
6. Sertakan identitas kelompok di bagian atas file (nama, NIM, kelas).
7. Kumpulkan hasil dalam bentuk:

   - File `.java`
   - Screenshot hasil eksekusi setiap operasi
   - Link GitHub/Replit/video demo
   - Laporan PDF singkat yang berisi analisis *time and growth rate* untuk setiap fungsi rekursif.
   - Pastikan semua berkas diunggah ke LMS.

BINUS UNIVERSITY ONLINE

People Innovation Excellence

Deskripsi Tugas

Kelompok diminta untuk mengembangkan sistem manajemen playlist musik yang telah dibuat pada tugas sebelumnya, dengan menambahkan kemampuan analisis rekursif dan pengukuran efisiensi algoritmik.

Program ini harus mampu:

1) Menghitung total durasi playlist menggunakan rekursi.
2) Menampilkan daftar lagu secara terbalik (reverse traversal) menggunakan rekursi.
3) Menentukan lagu dengan durasi terpanjang menggunakan rekursi.
4) Menganalisis kompleksitas waktu dan growth rate dari fungsi-fungsi tersebut.

Struktur Program yang Diharapkan

1. **Class Lagu**
   - Gunakan kembali dari Tugas Kelompok 1 (atribut: judul, artis, durasi).
2. **Class PlaylistRekursif**
   - Memiliki atribut Lagu[] playlist.
   - Memiliki fungsi-fungsi:
     - ✓ static double totalDurasi(Lagu[] list, int n);
     - ✓ static void tampilkanMundur(Lagu[] list, int index);
     - ✓ static double cariDurasiTerpanjang(Lagu[] list, int index);
   - Tambahkan pengukuran waktu eksekusi untuk masing-masing fungsi.

BINUS UNIVERSITY ONLINE

People Innovation Excellence

Soal:

1. Implementasi Fungsi Rekursif Dasar – (Bobot 40%)
   - `totalDurasi()` → menghitung total seluruh durasi lagu.
   - `tampilkanMundur()` → menampilkan lagu dari akhir ke awal.
   - `cariDurasiTerpanjang()` → mencari lagu dengan durasi terpanjang.

```
Total Durasi Lagu = 21.36 menit
Daftar Lagu (Terbalik):
Fix You - Coldplay (4.52)
Shivers - Ed Sheeran (3.50)
Perfect - Ed Sheeran (4.23)
Lagu Terpanjang: Fix You (4.52 menit)
```

2. Analisis Kompleksitas Waktu (Time and Growth Rate) – (Bobot 30%)

   Tuliskan analisis **Big-O** untuk setiap fungsi:

Fungsi	**Base Case**	**Growth Rate**	**Kompleksitas**
totalDurasi()	n == 0; return 0	Linear; C(n) = n + 1	Waktu O(n); ruang O(n)
tampilkanMundur()	n == 0; berhenti	Linear; C(n) = n + 1	Waktu O(n); ruang O(n)
cariDurasiTerpanjang()	n == 0; return 0	Linear; C(n) = n + 1	Waktu O(n); ruang O(n)

   Sertakan grafik sederhana (dalam laporan) yang menggambarkan *growth rate* linear terhadap ukuran data playlist (misal, n = 2, 4, 6, 8, 10).

3. Eksperimen Pengukuran Waktu Eksekusi (Bobot 20%)
   - Ukur dan tampilkan **waktu eksekusi (dalam milidetik)** untuk setiap fungsi.
   - Jalankan program dengan jumlah data berbeda (misal: 3, 5, dan 10 lagu).

- Bandingkan hasil waktu eksekusi dan buat tabel analisis di laporan:

Jumlah Lagu (n)	totalDurasi()	tampilkanMundur()	cariDurasiTerpanjang()
3	1 ms	1 ms	0 ms
5	2 ms	2 ms	1 ms
10	4 ms	4 ms	3 ms

4. Kerapian Program dan Dokumentasi (Bobot 10%)
   a. Kode tersusun rapi, menggunakan indentasi yang konsisten.
   b. Setiap fungsi utama memiliki komentar.
   c. Output program tampil jelas dan mudah dibaca.

Output yang diharapkan

```
=== ANALISIS REKURSIF PLAYLIST ===
Jumlah lagu : 5
Total durasi : 20.35 menit
Lagu terpanjang : "Viva La Vida" - Coldplay (4.52 menit)

Daftar lagu (ditampilkan terbalik):
5. Viva La Vida
4. Yellow
3. Fix You
2. Shivers
1. Perfect

Execution Time (totalDurasi): 3 ms
Execution Time (tampilkanMundur): 2 ms
```

```
Execution Time (cariDurasiTerpanjang): 3 ms
```

Anggota 4 :

Tabel hasil eksekusi :

```
=== PENGUKURAN WAKTU EKSEKUSI ===
Rata-rata waktu per pemanggilan (milidetik)
Jumlah Lagu  totalDurasi        tampilkanMundur    cariDurasiTerpanjang
3            0.000037           0.001812           0.000050
5            0.000018           0.001495           0.000032
10           0.000022           0.001661           0.000017
```

Jumlah Lagu	totalDurasi	tampilkanMundur	cariDurasiTerpanjang
3	0.000037	0.001812	0.000050
5	0.000018	0.001495	0.000032
10	0.000022	0.001661	0.000017

Analisis Waktu Eksekusi

Berdasarkan hasil eksperimen, pengukuran waktu eksekusi dilakukan terhadap tiga fungsi rekursif, yaitu totalDurasi(), tampilkanMundur(), dan cariDurasiTerpanjang(), menggunakan jumlah data sebanyak 3, 5, dan 10 lagu.

Fungsi totalDurasi() mencatat waktu sebesar 0.000037 ms untuk 3 lagu, 0.000018 ms untuk 5 lagu, dan 0.000022 ms untuk 10 lagu. Sementara itu, fungsi tampilkanMundur() mencatat waktu sebesar 0.001812 ms, 0.001495 ms, dan 0.001661 ms. Fungsi cariDurasiTerpanjang() mencatat waktu sebesar 0.000050 ms, 0.000032 ms, dan 0.000017 ms secara berurutan.

Hasil pengukuran menunjukkan bahwa waktu eksekusi tidak selalu meningkat ketika jumlah lagu bertambah. Hal ini dapat dipengaruhi oleh ukuran data yang relatif kecil, optimasi JVM, dan overhead pengukuran. Pada fungsi tampilkanMundur(), waktu pengukuran juga mencakup overhead operasi pencetakan ke output kosong.

Secara teoretis, ketiga fungsi rekursif memiliki kompleksitas waktu O(n), karena setiap pemanggilan memproses satu lagu dan melanjutkan rekursi dengan ukuran masalah yang berkurang satu.

BINUS UNIVERSITY ONLINE

People Innovation Excellence

Analisis Kompleksitas dan Dokumentasi Playlist Rekursif

Anggota 5: Khenaro Daffa Asyrof | NIM 2902824590 | Kelompok 5 | LYCA-LEC

1. Analisis Kompleksitas Waktu dan Ruang

Ketiga fungsi memproses satu lagu pada setiap langkah, kemudian melanjutkan ke n - 1 lagu. Relasi waktunya T(n) = T(n - 1) + c dengan T(0) = c0. Setelah diuraikan, T(n) = c0 + n c, sehingga pertumbuhannya linear: Θ(n), dengan batas atas O(n).

Parameter n pada salinan program menyatakan jumlah lagu yang diperiksa, bukan indeks terakhir. Pemanggilan awal menggunakan n = jumlahLagu; akses lagu dilakukan lewat list[n - 1]. Analisis berlaku untuk array tidak null, $0 \leq n \leq$ list.length, seluruh elemen pada rentang tersebut terisi, dan durasi tidak negatif.

1.1 Fungsi totalDurasi

Saat n = 0, fungsi mengembalikan 0. Untuk n > 0, durasi list[n - 1] dijumlahkan dengan totalDurasi(list, n - 1). Tidak ada percabangan yang melewati sisa data, sehingga semua n lagu tetap diperiksa. Terdapat n operasi penjumlahan dan n + 1 pemanggilan, termasuk base case. Waktunya Θ(n); ruang tambahan call stack Θ(n).

1.2 Fungsi tampilkanMundur

Saat n = 0, fungsi berhenti. Pada langkah rekursif, lagu berindeks n - 1 dicetak terlebih dahulu, lalu fungsi dipanggil untuk n - 1 lagu. Dengan n = 10, urutannya adalah indeks 9 sampai 0. Fungsi mencetak n baris dan melakukan n + 1 pemanggilan. Dengan panjang judul dan artis dibatasi, waktu dan call stack masing-masing Θ(n). Bila panjang teks ikut dihitung, biaya pencetakan bergantung pula pada total karakter keluaran.

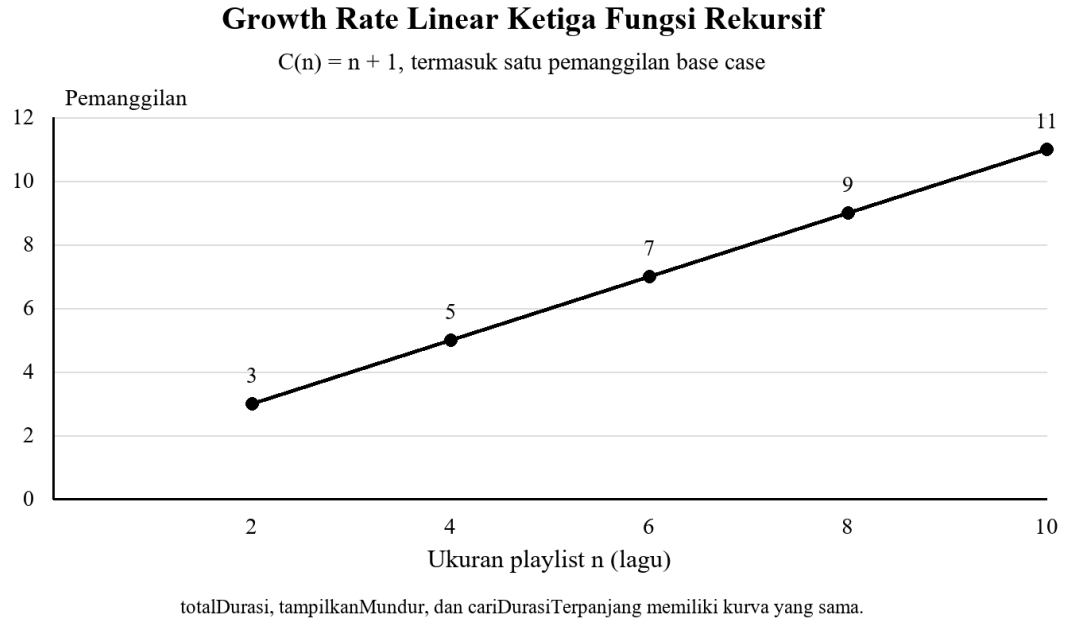
1.3 Fungsi cariDurasiTerpanjang

Salinan sesuai instruksi mengembalikan double. Base case n = 0 menghasilkan 0; langkah rekursif memakai Math.max antara durasi list[n - 1] dan hasil untuk n - 1 lagu. Semua n lagu diperiksa, sehingga waktu dan call stack masing-masing Θ(n). Nama lagu ditentukan terpisah di main untuk keperluan tampilan; pencarian tambahan tersebut juga O(n) dan tidak mengubah kompleksitas fungsi rekursif.

Untuk input kosong, masing-masing fungsi menyelesaikan base case dalam O(1). Untuk n yang bertambah, ketiga fungsi tidak memiliki penghentian awal. Analisis ruang di atas tidak memasukkan array playlist yang sudah disediakan.

2. Grafik Growth Rate Linear

Grafik menggunakan ukuran playlist n = 2, 4, 6, 8, dan 10. Sumbu vertikal menyatakan jumlah pemanggilan rekursif C(n) = n + 1. Ketiga kurva berimpit karena seluruh fungsi memiliki satu langkah rekursif untuk setiap lagu dan satu pemanggilan terakhir saat n = 0. Grafik ini adalah model teoretis, bukan hasil pengukuran milidetik.



Gambar 1. Growth rate linear ketiga fungsi berdasarkan jumlah pemanggilan.

n	totalDurasi	tampilkanMundur	cariDurasiTerpanjang
2	3	3	3
4	5	5	5
6	7	7	7
8	9	9	9
10	11	11	11

Dasar penghitungannya adalah C(0) = 1 dan C(n) = 1 + C(n - 1). Jika C(k) = k + 1, maka C(k + 1) = 1 + C(k) = k + 2; jadi C(n) = n + 1. Penambahan dua lagu menambah dua pemanggilan. Konstanta biaya tiap fungsi berbeda, tetapi orde pertumbuhannya tetap linear.

3. Integrasi Hasil Pengukuran Waktu

Eksperimen Anggota 4 menggunakan System.nanoTime(), 10.000 pengulangan per ukuran data, dan konversi nanodetik ke milidetik. Data yang diserahkan Anggota 4 dicantumkan kembali agar sumber hasil eksperimen jelas.

n	totalDurasi (ms)	tampilkanMundur (ms)	cariDurasiTerpanjang (ms)
3	0.000037	0.001812	0.000050
5	0.000018	0.001495	0.000032
10	0.000022	0.001661	0.000017

Tabel 1. Hasil Anggota 4 pada kode yang diserahkan kelompok.

Uji ulang berikut dilakukan pada salinan yang telah disesuaikan agar cariDurasiTerpanjang mengembalikan double. Lingkungan pengujian: Windows, OpenJDK 21.0.10, 18-10-2026 21:24:11 WIB. Hasilnya tidak menggantikan tabel Anggota 4.

n	totalDurasi (ms)	tampilkanMundur (ms)	cariDurasiTerpanjang (ms)
3	0.000127	0.010360	0.000302
5	0.000063	0.009449	0.000187
10	0.000064	0.010626	0.000123

Tabel 2. Hasil uji ulang salinan sesuai instruksi; rata-rata per pemanggilan.

Waktu dihitung sebagai (selesai - mulai) / 1.000.000 / 10.000. Hasil yang sangat kecil dapat tampil 0 ms jika dibulatkan ke bilangan bulat; hal itu tidak berarti fungsi tidak membutuhkan waktu. Enam angka desimal digunakan untuk mempertahankan hasil kecil tersebut.

Nilai tidak selalu naik ketika n bertambah karena ukuran data kecil, pemanasan dan optimasi JVM, serta overhead pengukuran. Hasil numerik fungsi total dan maksimum tidak dikonsumsi dalam loop, sehingga optimasi JVM juga dapat memengaruhinya. Pada tampilkanMundur, pembentukan teks dan pemanggilan println tetap dijalankan meskipun output dialihkan ke stream kosong. Data ini merupakan eksperimen sederhana, bukan benchmark terkontrol, dan tidak digunakan sebagai bukti tunggal Big-O.

4. Dokumentasi Hasil Eksekusi

Program dikompilasi menggunakan javac -encoding UTF-8. Gambar berikut merupakan screenshot halaman dokumentasi lokal yang menampilkan keluaran proses Java aktual. Tampilan tersebut bukan IDE; teks keluaran berasal dari program yang dijalankan tanpa mengganti hasilnya.

Pada 10 lagu, total durasi adalah 42,22 menit dan durasi terbesar 7,29 menit, yaitu IN THE NAME OF LOVE. Nilai durasi pada kode diperlakukan sebagai menit desimal, bukan format menit:detik.



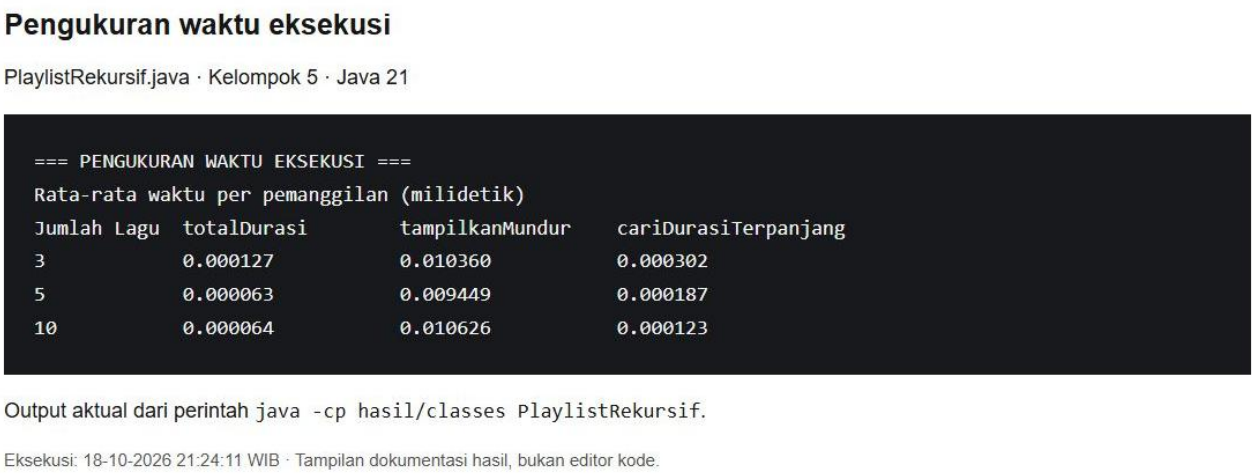
Gambar 2. Hasil total durasi dan lagu dengan durasi terpanjang.



Gambar 3. Reverse traversal menghasilkan urutan lagu 10 sampai 1.

5. Verifikasi dan Reproduksi

Pengujian otomatis membandingkan total dan maksimum dengan perhitungan iteratif, lalu memeriksa jumlah baris dan urutan reverse traversal. Ukuran yang diuji adalah 0, 1, 2, 3, 4, 5, 6, 8, dan 10 lagu, ditambah kasus dua durasi sama. Seluruh 69 pemeriksaan lulus.



Pengukuran waktu eksekusi

PlaylistRekursif.java · Kelompok 5 · Java 21

```
=== PENGUKURAN WAKTU EKSEKUSI ===
Rata-rata waktu per pemanggilan (milidetik)
Jumlah Lagu  totalDurasi        tampilkanMundur    cariDurasiTerpanjang
3            0.000127           0.010360           0.000302
5            0.000063           0.009449           0.000187
10           0.000064           0.010626           0.000123
```

Output aktual dari perintah `java -cp hasil/classes PlaylistRekursif`.

Eksekusi: 18-10-2026 21:24:11 WIB · Tampilan dokumentasi hasil, bukan editor kode.

Gambar 4. Pengukuran waktu aktual pada salinan program sesuai instruksi.

6. Berkas dan Catatan Implementasi

Buka folder source, kompilasi dengan javac -encoding UTF-8 PlaylistRekursif.java, lalu jalankan java PlaylistRekursif. File PlaylistRekursifTest.java hanya untuk pengujian tambahan, bukan kebutuhan runtime.

Salinan asli tersimpan pada source_asli. Perbaikan bertanda - khenaro menyesuaikan return type maksimum menjadi double, pemanggil di main, dan komentar parameter. Data lagu serta logika pengukuran Anggota 4 dipertahankan.

Ketiga fungsi memiliki waktu O(n) dan ruang tambahan O(n). Waktu aktual dipengaruhi konstanta dan kondisi runtime. Rekursi pada playlist sangat besar berisiko melampaui batas call stack.

Referensi

Oracle. Java SE 21 API: System.nanoTime().

Kelompok 5. PlaylistRekursif (4).java dan tabel pengukuran Anggota 4 pada R1_TK3.

Repository GitHub: playlist-rekursif-kelompok-5.